

# Grundlagen der C-Programmierung

Dr. Henrik Brosenne  
Georg-August Universität Göttingen  
Institut für Informatik

Wintersemester 2024/25

## Strukturen

- Typnamen

- Vereinbarung von Strukturen

- Operationen mit Strukturen

- Strukturen auf dem Heap

- Verkettete Listen

- Verbunde

# Typnamen

Das Schlüsselwort `typedef` erlaubt die Deklaration eigener Typnamen.

```
typedef typ name;
```

Üblich ist es, für Typnamen nur Großbuchstaben zu verwenden oder ihnen `_t` anzufügen. Diese beiden Varianten werden auch von der Standardbibliothek verwendet.

## Beispiel

---

```
1  typedef int INTEGER;
2  typedef enum color color_t;
3  typedef enum { OFF = 0, ON } FLAG;
4
5  INTEGER i = 1;
6  color_t chalk;
7  FLAG check;
```

---

Durch `typedef` werden keine neuen Typen deklariert, sondern nur Typen benannt, dadurch kann die Lesbarkeit des Quelltextes verbessert werden.

## Strukturen

Typnamen

**Vereinbarung von Strukturen**

Operationen mit Strukturen

Strukturen auf dem Heap

Verkettete Listen

Verbunde

# Vereinbarung von Strukturen

Häufig ist es wünschenswert, *Objekte* mit unterschiedlichem Typ zu logischen Einheiten zusammenzufassen, das geht in C mit **Strukturen**.

Will man etwa eine Studentendatei verwalten, benötigt man (u.a.) den Namen, die Matrikelnummer und die Adresse jedes Studenten.

Die Deklaration eines Strukturtyps beginnt mit dem Schlüsselwort **struct**.

Ihm kann ein Name folgen, mit dem dann im weiteren Programm der Strukturtyp bezeichnet werden kann.

Ihm folgt, in geschweifte Klammern eingeschlossen, die Aufzählung der Komponenten des Strukturtyps.

Abgeschlossen wird die Deklaration durch ein Semikolon:

```
struct [name] {  
    typ komponente1;  
    typ komponente2;  
    ...  
    typ komponenteN;  
};
```

# Beispiel, Studentendaten

---

```
1 struct student_s {
2     char first_name[30];
3     char last_name[30];
4     int matric_no;
5     int postal_code;
6     char city[30];
7     char street_name[30];
8     int street_number;
9 };
```

---

Die Gültigkeit der Namen der Komponenten eines Strukturtyps ist auf den Strukturtyp beschränkt. Damit entstehen auch dann keine Namenskonflikte, wenn eine Komponente denselben Namen wie eine Variable, ein Feld, eine Funktion oder auch eine Komponente eines anderen Strukturtyps besitzt.

In der beschriebenen Form handelt es sich um eine reine Deklaration, d.h. es wird noch keine Variable mit dem Typ definiert und entsprechend kein Speicherplatz bereitgestellt.

# Definition

Variablen mit einem Strukturtyp (kurz Strukturen), kann man auf verschiedene Weisen definieren.

Zunächst kann man die Namen direkt zwischen die schließende geschweifte Klammer und das nachfolgende Semikolon der Typdeklaration setzen.

---

```
1 struct student_s {  
2     char first_name[30];  
3     // ... as above  
4 } student1, student2;
```

---

Will man im Rest des Programms keinen Bezug mehr auf den Strukturtyp nehmen, kann man den Namen, hier also **student\_s**, auch weglassen.

Ist der Name angegeben, kann man Strukturen auch wie folgt definieren.

---

```
1 struct student_s student1, student2;
```

---

# typedef

Für die Lesbarkeit eines Programms ist es oft günstiger Strukturtypen mit `typedef` Namen zu geben.

---

```
1 typedef struct student_s {
2     char first_name[30];
3     // ... as above
4 } student_t;
```

---

Jetzt kann man Strukturen z.B. wie folgt definieren.

---

```
1 struct student_s student1, student2;
2 student_t student3, student4;
```

---

Bei der Deklaration kann man den Strukturnamen `student_s` weglassen, dann kann man aber nur noch den `typedef`-Namen `student_t` verwenden.

Eine Kombination von `typedef`-Deklaration und Strukturdefinition ist **nicht** möglich.

Das nachgestellten `_s` bzw. `_t` ist nicht zwingend erforderlich, verdeutlicht aber, dass es sich um einen Strukturnamen bzw. einen `typedef`-Namen handelt.



## Strukturen

Typnamen

Vereinbarung von Strukturen

**Operationen mit Strukturen**

Strukturen auf dem Heap

Verkettete Listen

Verbunde

# Operationen mit Strukturen (1/2)

Operationen mit Strukturen als Ganzem sind nur sehr eingeschränkt möglich.

- Wertzuweisungen zwischen Strukturen sind möglich.

Dabei werden die Werte aller Komponenten übertragen. Offensichtliche Voraussetzung ist die Identität der Strukturtypen.

- Funktionen können Strukturen als Parameter besitzen.

Die Abarbeitung der Aufrufe erfolgt wie bei Variablen. Im Zuge des Aufrufs wird der Wert des Arguments in eine lokale Struktur der Funktion kopiert.

Zumindest bei umfangreicheren Strukturen sollte man sich also überlegen, ob die Übergabe eines Zeigers auf die Struktur nicht zweckmäßiger ist.

- Funktionen können Strukturen als Funktionswerte zurückliefern.

Wie bei den Parametern sollte man sich überlegen, ob das zweckmäßig ist. Auch hier kann ein Zeiger-Parameter die bessere Lösung sein.

- Mit dem Adressoperator & kann man die Adresse einer Struktur ermitteln.

# Operationen mit Strukturen

Mit den Komponenten von Strukturen kann man wie mit Variablen arbeiten.

Allerdings reicht die Angabe des Namens einer Komponente nicht aus, da alle Strukturen mit gleichem Typ gleichnamige Komponenten besitzen. Erforderlich ist, ähnlich wie bei Zeigern, eine Dereferenzierung.

Angegeben werden der Name der Struktur **und** der Name der Komponente; zwischen beide wird der **Punktoperator** (.) gesetzt.

---

```
1 struct student_s student;  
2 ...  
3 student.postal_code = 11011;
```

---

# Datensatz eines Studenten Ein- und Ausgeben (1/2)

```
1  #include <stdio.h>
2
3  //*****
4  typedef struct {
5      char first_name[30];
6      char last_name[30];
7      int matric_no;
8  } student_t;
9
10 //*****
11 student_t scan_record();
12 void print_record(student_t s);
13
14 //=====
15 int main() {
16     student_t data;
17
18     printf("enter data set\n");
19     data = scan_record();
20     printf("data set\n");
21     print_record(data);
22
23     return 0;
24 }
```

## Datensatz eines Studenten Ein- und Ausgeben (2/2)

```
26 //=====
27 student_t scan_record() {
28     student_t s;
29
30     printf("%%first_name:");
31     scanf("%s", s.first_name);
32     printf("%%last_name:");
33     scanf("%s", s.last_name);
34     printf("%%matriculation_number:");
35     scanf("%d", &s.matric_no);
36
37     return s;
38 }
39
40 //=====
41 void print_record(student_t s) {
42     printf("%%first_name:%s\n", s.first_name);
43     printf("%%last_name:%s\n", s.last_name);
44     printf("%%matriculation_number:%d\n", s.matric_no);
45 }
```

# Zeiger auf Strukturen

Für Zeiger auf Strukturen gelten dieselben Regeln wie für Zeiger auf einfache Variablen.

---

```
1 struct student_s student, *p = &student;
```

---

Beim Zugriff auf die Komponenten muss man berücksichtigen, dass der Punktoperator höhere Priorität besitzt als der Dereferenzierungsoperator (\*).

Man **muss** also Klammern.

---

```
1 (*structure_pointer).structure_component
```

---

Abkürzend und suggestiver ist die äquivalente Schreibweise mit dem Pfeil-Operator (->).

---

```
1 structure_pointer->structure_component
```

---

## Strukturen

Typnamen

Vereinbarung von Strukturen

Operationen mit Strukturen

**Strukturen auf dem Heap**

Verkettete Listen

Verbunde

# Strukturen auf dem Heap (1/2)

Für Zeiger auf Strukturen gelten dieselben Regeln wie für Zeiger auf einfache Variablen.

Definiert man eine Zeigervariable, so wird nur der Speicher für die Aufnahme des Zeigers bereitgestellt. Durch Zuweisung eines Zeigers auf eine statische oder automatische Struktur kann man solch einer Zeigervariablen einen Wert geben.

Bereitstellung von Speicher auf dem Heap ist möglich, hier ist der Operator **sizeof** unentbehrlich.

---

```
1  struct student_s *p;  
2  ...  
3  p = malloc(sizeof(struct student_s));
```

---



# Strukturen auf dem Heap (2/2)

---

```
1  student_t *p;  
2  ...  
3  p = malloc(sizeof(student_t));
```

---

Wieviel Speicher eine Struktur benötigt, kann von Rechner zu Rechner unterschiedlich sein.

Dahinter steht folgende Vorschrift des Standards.

Die Komponenten einer Struktur müssen im Speicher des Rechners zwar dieselbe Reihenfolge besitzen wie in der Deklaration – sie müssen aber nicht lückenlos aufeinanderfolgen.

Entsprechend addiert der Operator `sizeof` nicht einfach den Speicherbedarf der Komponenten, sondern berücksichtigt auch eventuelle Lücken.

Als Grundregel gilt: Bei jedem Aufruf von `malloc` sollte im Argument der Operator `sizeof` verwendet werden.

## Strukturen

Typnamen

Vereinbarung von Strukturen

Operationen mit Strukturen

Strukturen auf dem Heap

**Verkettete Listen**

Verbunde

# Verkettete Listen

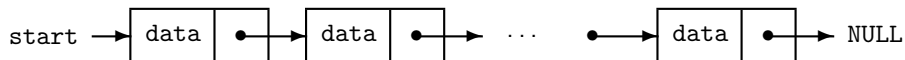
Strukturen und Zeiger auf Strukturen bilden die Grundlage einer wichtigen nicht elementaren Datenstruktur, nämlich der **verketteten Liste**.

Anschaulich kann man verkettete Listen mit „Schnitzeljagd“ vergleichen:

- Man hat einen Verweis, wo man beginnen muss (Zeigervariable).
- Am Start findet man einen Zettel mit einer Aufgabe und einen neuen Verweis (Struktur mit Daten- und Zeigerkomponente).
- Folgt man den Verweisen, kommt man irgendwann zum Ende (Zeigerkomponente mit Nullzeiger).

# Aufbau

## Beispiel



In einem Programm kann man das durch Deklarationen wie die nachfolgende Erreichen.

---

```
1 typedef struct list_s {
2     data_t data;
3     struct list_s *next;
4 } list_t;
```

---

Es handelt sich scheinbar um eine „rekursive Deklaration“, aber tatsächlich ist die zweite Komponente ein Zeigertyp und steht nur für eine Adresse, nicht für den Strukturtyp, der gerade deklariert wird.

# Standardoperationen

Standardoperationen für verkettete Listen sind

- Einfügen eines neuen Eintrags.
- Suchen eines Eintrags mit bestimmtem Wert.
- Entfernen eines Eintrags.

Typischerweise werden verkettete Listen auf dem Heap angelegt.

Der Fantasie sind beim Aufbau verketteter Strukturen keine Grenzen gesetzt.

- In einer verketteten Liste kann jeder Eintrag Anfang einer verketteten Liste sein.
- Mehrere Zeigerkomponenten in jeder Struktur erlauben den Aufbau von mehrfach verketteten Listen oder auch von **Bäumen**.

# Beispiel (1/4)

## Aufgabe

Erstellen einer (einfach) verketteten Liste für ganze Zahlen. Einfügen von N Zahlen in die Liste und Ausgeben der Liste.

---

```
4  #include <stdio.h>
5  #include <stdlib.h>
6
7  #define N 10
8
9  // *****
10 typedef struct list_s {
11     int value;
12     struct list_s *next;
13 } list_t;
14
15 // *****
16 list_t *list_append(list_t *list, int value);
17 void list_print(list_t *list);
18 void list_free(list_t *list);
```

---

## Beispiel (2/4)

---

```
10 typedef struct list_s {
11     int value;
12     struct list_s *next;
13 } list_t;
```

---

---

```
38 //=====
39 list_t *list_append(list_t *list, int value) {
40     list_t *node = malloc(sizeof(list_t));
41     if (node == NULL)
42         return list;
43     node->value = value;
44     node->next = list;
45     return node;
46 }
```

---

## Beispiel (3/4)

```
10 typedef struct list_s {
11     int value;
12     struct list_s *next;
13 } list_t;
```

```
48 //=====
49 void list_free(list_t *list) {
50     list_t *p;
51     while (list != NULL) {
52         p = list;
53         list = list->next;
54         free(p);
55     }
56 }
57
58 //=====
59 void list_print(list_t *list) {
60     if (list == NULL) {
61         printf("NULL\n");
62         return;
63     }
64     printf("%d_>_", list->value);
65     list_print(list->next);
66 }
```



## Beispiel (4/4)

---

```
20 //=====
21 int main() {
22     list_t *list = NULL, *p;
23
24     for(int i = 0; i < N; i++) {
25         if ((p = list_append(list, i)) == list) {
26             printf("ERROR: list_append\n");
27             list_free(list);
28             return 0;
29         }
30         list = p;
31     }
32
33     list_print(list);
34     list_free(list);
35     return 0;
36 }
```

---

## Strukturen

Typnamen

Vereinbarung von Strukturen

Operationen mit Strukturen

Strukturen auf dem Heap

Verkettete Listen

**Verbunde**

# Verbunde

Formal den Strukturen sehr ähnlich sind die **Verbunde**.

Statt **struct** schreibt man **union**. Inhaltlich bestehen jedoch ganz gravierende Unterschiede.

- Bei Strukturen belegen aufeinanderfolgende Komponenten aufeinanderfolgende Plätze im Speicher, wenn auch vielleicht nicht lückenlos, wie wir gesehen haben.
- Bei einem Verbund beginnen alle Komponenten an derselben Speicheradresse. Der Verbund belegt insgesamt so viel Speicher wie seine längste Komponente. Dadurch wird erreicht, dass der gleiche Speicherplatz zu unterschiedlichen Zeitpunkten in unterschiedlicher Weise genutzt werden kann, z.B. je nach Situation zur Speicherung eines **float**-Wertes oder (wenn dieser gerade nicht benötigt wird) zur Speicherung eines **int**-Wertes.

# Verbunde

In der Praxis kommen Verbunde recht selten vor. Ein kurzes Beispiel soll zeigen, dass ihre Verwendung extrem leicht zu Fehlern führt.

Man kann sich mit Castoperatoren selber „austricksen“.

---

```
1  int i, *ip = &i;
2  float *fp;
3  //...
4  fp = (float *) ip;
5  *fp = 6.5f;
6  printf("%d\n", i);
```

---

Mit einem Verbund kann man dasselbe noch einfacher haben.

---

```
1  union {
2      int i;
3      float f;
4  } nonsense;
5  //...
6  nonsense.f = 6.5f;
7  printf("%d\n", nonsense.i);
```

---